

Public Key Infrastructure and the Full Picture

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



August 7, 2026

Table of Content

- 1 Communication when two parties
- 2 Agreeing on Secret
- 3 Encrypting the secret
- 4 RSA
- 5 ElGamal
- 6 Publishing Public-key
- 7 Digital Signature
- 8 Revisiting the Full Picture

Recommended Reading From textbook: Chapter *

Communication when two parties have shared secret

From Session Key to Secure Communication

Once a shared secret has been established, both communicating parties derive a common **session key** (short secret).



Using the Session Key

- **Confidentiality:** Block ciphers or stream ciphers.
- **Integrity:** Public cryptographic hash functions.
- **Message Authentication:** Keyed cryptographic hash functions (MACs) or authenticated encryption.
- For every message, a fresh **IV/Nonce** is generated using a predetermined algorithm (e.g., random generation or a monotonically increasing counter) and transmitted with the ciphertext, with authenticated by **AEAD**.

Observation

The session key remains fixed throughout the session, whereas the IV/Nonce is unique for every message encrypted under that session key.

From Hash Functions to Message Authentication Codes (MACs)

A cryptographic hash function is public. Therefore, a naive construction

$$H(K \| m)$$

is **not** a secure message authentication code.

Instead, standardized constructions such as **HMAC** are used.

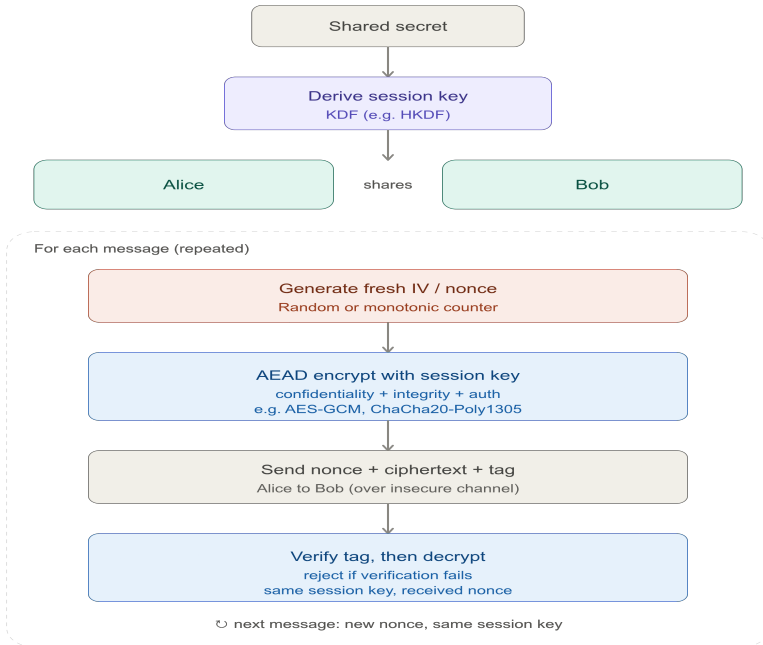
$$\text{MAC} = \text{HMAC-SHA256}(K, m).$$

The HMAC construction is

$$\text{HMAC}(K, m) = H((K \oplus \text{opad}) \| H((K \oplus \text{ipad}) \| m)).$$

Applications

- Message authentication without encryption.
- API authentication and software integrity.
- Authentication of protocol metadata (e.g., IV/nonce, packet headers, sequence numbers) when they are transmitted in plaintext.



How to agree on a shared secret?

The Key Agreement Problem

Almost every cryptographic primitive studied so far assumes that both parties already share a secret key.



The fundamental question is

How can two strangers establish a common secret over a public channel?

Key Agreement Protocol

A protocol that enables two parties to derive the same secret key without transmitting the key itself.

Public-Key Encryption vs. Key Agreement

Two approaches exist for establishing shared secrets.

Public-Key Encryption

Alice generates a random key K and encrypts it using Bob's public key.

$$K \longrightarrow \text{Enc}_{pk}(K)$$

Only Bob can recover K .

Key Agreement

Alice and Bob each contribute fresh randomness.
Neither party chooses the final key.
Both compute

$$K = g^{ab}.$$

Diffie-Hellman is the most celebrated key agreement protocol.

Note

Public-key Encryption Only method is outdated, as only one person determines secret.

Diffie-Hellman Key Exchange (DHKE)

Public parameters

$$(G, q, g),$$

where

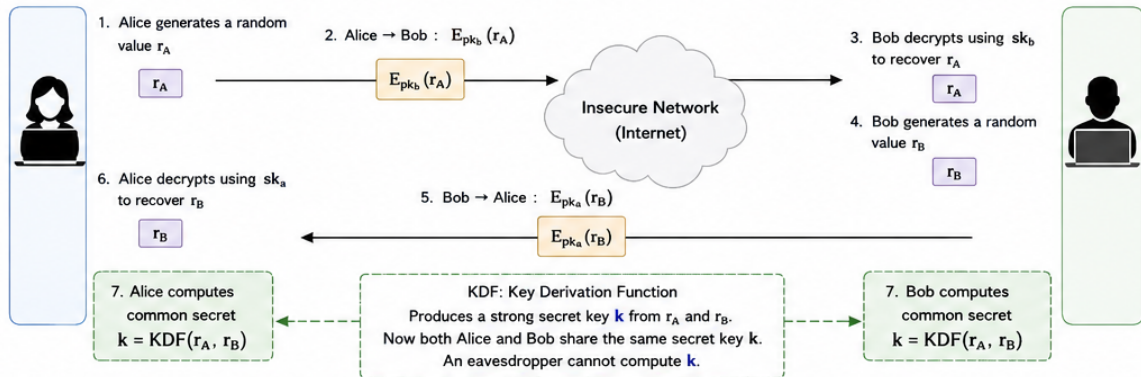
- G is a cyclic group,
- $|G| = q$,
- g is a generator of G .

Alice	Bob
$a \xleftarrow{\$} \mathbb{Z}_q$	$b \xleftarrow{\$} \mathbb{Z}_q$
$A = g^a$	$B = g^b$
$A \rightarrow$	$\leftarrow B$
$K = B^a$	$K = A^b$

Both compute shared secret

$$K = g^{ab}.$$

Establishing a Common Secret



Shared Secret vs. Session Key

The shared secret

$$K = g^{ab}$$

is generally *not* used directly as the encryption key. Instead, both parties apply a **Key Derivation Function (KDF)** to derive a key of the required length.

$$K_{\text{sess}} = \text{KDF}(K)$$

For example, one may use

$$K_{\text{sess}} = \text{SHA256}(K)$$

for illustration, while practical systems typically use standardized KDFs such as HKDF.

Why Does DHKE Work?

Correctness follows from

$$(B)^a = (g^b)^a = g^{ab} = (g^a)^b = (A)^b.$$

Security Assumption

Given

$$(g, g^a, g^b),$$

an adversary should not be able to compute

$$g^{ab}.$$

This is known as the **Computational Diffie-Hellman (CDH) Assumption**.

The underlying hardness comes from the Discrete Logarithm Problem (DLP).

Choosing the Group

The security of DHKE depends entirely on the choice of the group.

Group	Assumption
$\mathbb{Z}_p^*$	DLP
Elliptic Curve Groups	ECDLP
Class Groups	Class Group DLP

The group should satisfy

- Efficient group operations.
- Efficient exponentiation.
- Computationally hard Discrete Logarithm Problem.
- No known efficient classical attacks.

Typical choices include

- Multiplicative group $\mathbb{Z}_p^*$,
- Elliptic curve groups.

Observation

The same protocol can be instantiated over different algebraic groups, provided the DLP remains computationally hard.

Examples of Classical Key Agreement Protocols

Most classical key exchange protocols rely on the hardness of the Discrete Logarithm Problem.

Protocol	Underlying Assumption
Diffie-Hellman (DH)	DLP / CDH
Elliptic Curve DH (ECDH)	Elliptic Curve DLP
MQV / HMQV	DLP
SIGMA	DLP + Digital Signatures
TLS 1.2 (ECDHE)	Elliptic Curve DLP

Other key establishment mechanisms are based on different assumptions.

RSA $\rightarrow$ Integer Factorization

Where is Diffie-Hellman Used?

Today, Diffie-Hellman is deployed in numerous Internet protocols.

Protocol	Key Agreement
TLS 1.2	ECDHE
TLS 1.3	ECDHE
SSH	Diffie-Hellman / ECDH
Signal	X25519
WireGuard	Curve25519

Most modern deployments use elliptic-curve Diffie-Hellman.

The Quantum Challenge

Classical key agreement protocols rely on mathematical assumptions such as

- Discrete Logarithm Problem,
- Integer Factorization.

Large-scale quantum computers can solve both problems efficiently using

Shor's Algorithm.

Consequently,

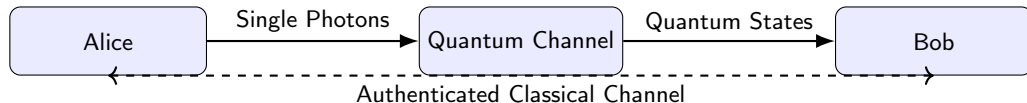
- Diffie-Hellman becomes insecure,
- ECDH becomes insecure,
- RSA key exchange becomes insecure.

Next Step

Can we design key agreement protocols that remain secure even against quantum computers?

Solution 1: Quantum Key Distribution (QKD)

Instead of relying on computational hardness assumptions, Quantum Key Distribution (QKD) derives its security from the laws of quantum mechanics.



Idea

Alice encodes random bits into quantum states. Bob measures them. Any eavesdropping disturbs the quantum states and is therefore detectable.

Limitations of QKD

Although QKD provides information-theoretic security, practical deployment remains challenging.

- Requires dedicated quantum communication channels.
- Requires specialized quantum hardware.
- Limited transmission distance without quantum repeaters.
- Expensive deployment and maintenance.
- Still requires an authenticated classical communication channel.

Observation

QKD is suitable for specialized high-security applications, but is currently impractical for securing the global Internet.

Solution 2: Post-Quantum Key Exchange

Instead of changing the communication medium, we redesign the cryptographic algorithms.

Classical Network + New Mathematics

Post-Quantum Cryptography (PQC)

- Runs on existing computers.
- Uses existing Internet infrastructure.
- Resists attacks from both classical and quantum computers.
- Requires no quantum communication channel.

Modern standards replace Diffie-Hellman by **Post-Quantum Key Encapsulation Mechanisms (KEMs)**.

Hardness Assumptions for Post-Quantum Cryptography

Unlike classical cryptography, PQC relies on mathematical problems for which no efficient classical or quantum algorithms are currently known.

Family	Representative Assumption
Lattice-based	LWE, MLWE, SIS
Code-based	Syndrome Decoding
Multivariate	MQ Problem
Hash-based	Hash Function Security
Isogeny-based	Isogeny Problems

Today

The NIST post-quantum standards primarily rely on **lattice-based cryptography**.

What If a Hardness Assumption Is Broken?

Suppose a future breakthrough shows that a widely used hardness assumption (e.g., lattice problems) can be solved efficiently.

Immediate Consequence

All cryptographic primitives whose security relies on that assumption would need to be replaced.

Fortunately, post-quantum cryptography is based on multiple independent families of hard problems.

Family	Example
Code-based	McEliece
Hash-based	XMSS, SPHINCS ⁺
Multivariate	MQ-based Schemes
Isogeny-based	SIKE (Research)
Lattice-based	ML-KEM, ML-DSA

Future Directions

- Develop cryptography from new mathematical assumptions.
- Explore alternative computational paradigms (e.g., DNA computing, molecular cryptography, etc.).
- Continuously analyze, standardize, and diversify cryptographic assumptions.

A Natural Question

So far we assumed that Alice already knows Bob's public key.

For example,

$$\text{Enc}_{pk_B}(r_A),$$

or

$$\text{Verify}_{pk_B}(\sigma).$$

But this raises an important question.

Where Do Public Keys Come From?

How does Alice obtain Bob's authentic public key?

How does Bob obtain Alice's public key?

If an attacker replaces a public key with its own, all subsequent encryption and signatures become meaningless.

This is known as the **Public Key Distribution Problem**.

Towards Public Key Infrastructure (PKI)

Public keys themselves must be authenticated before they can be trusted.

This is achieved using

- Digital Certificates,
- Certificate Authorities (CA),
- Public Key Infrastructure (PKI),
- Certificate Chains.

Trust the Certificate $\implies$ Trust the Public Key

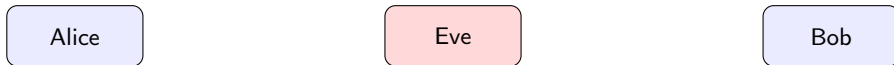
Next Topic

How are public keys certified and distributed securely over the Internet?

If everything depends on public keys, who authenticates the public keys?

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.

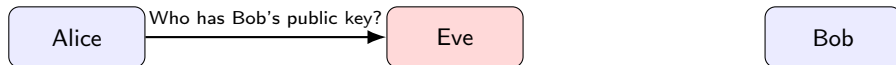


Alice wants to communicate securely with Bob.

Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Alice asks for Bob's public key.

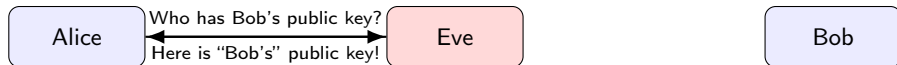
Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Eve intercepts the request and sends her own public key.

Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Alice unknowingly encrypts the session key using Eve's public key.

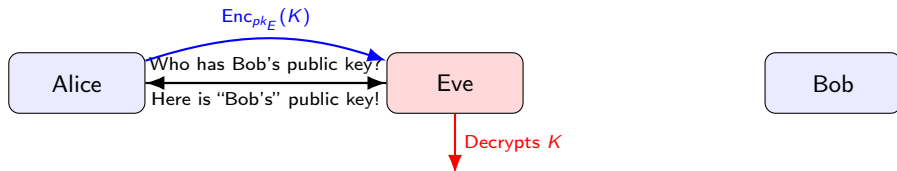
Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Eve decrypts the session key and learns the secret.

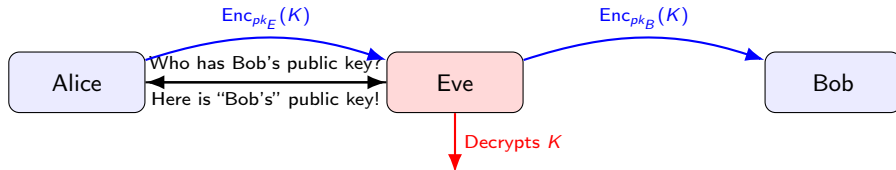
Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Eve re-encrypts the same session key using Bob's public key and forwards it. Bob believes the key came directly from Alice.

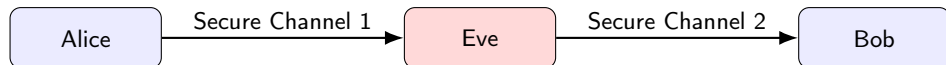
Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Man-in-the-Middle (MITM) Attack

Without authenticated public keys, an attacker can establish two independent secure channels.



Eve decrypts every message, reads or modifies it, re-encrypts it, and forwards it to Bob. Neither Alice nor Bob realizes that the communication is being intercepted.

Solution

Authenticate public keys using digital certificates and a trusted Public Key Infrastructure (PKI).

Encrypting the secret

Why Do We Need Public-Key Encryption?

Suppose Alice and Bob wish to establish a shared secret.

They independently generate random values a, b .

These values must be transmitted over an insecure communication channel.

Can We Use a Block or Stream Cipher?

No. Block and stream ciphers are **symmetric** primitives. They require a pre-established secret key K , before encryption can even begin.

Thus, they cannot solve the initial key establishment problem.

Solution

Use the recipient's **public key**.

For example, Alice sends

$$c = \text{Enc}_{pk_B}(a),$$

and only Bob can recover

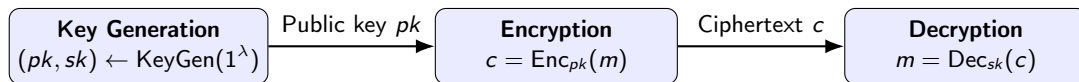
$$a = \text{Dec}_{sk_B}(c).$$

Similarly, Bob encrypts b using Alice's public key.

Generic Public-Key Encryption Scheme

A public-key encryption scheme consists of three polynomial-time algorithms

(KeyGen, Enc, Dec).



Key Idea

- The public key pk is distributed openly.
- Anyone can encrypt using pk .
- Only the owner of the secret key sk can decrypt.

Key-Pair Generation RSA

RSA: Key Generation

RSA is based on the hardness of the **Integer Factorization Problem**.

- 1 Choose two large primes

$$p, q.$$

- 2 Compute

$$N = pq, \quad \phi(N) = (p-1)(q-1).$$

- 3 Choose

$$e \text{ such that } \gcd(e, \phi(N)) = 1.$$

- 4 Compute

$$d \equiv e^{-1} \pmod{\phi(N)}.$$

$$pk = (N, e), \quad sk = (N, d).$$

RSA Encryption and Decryption

Encryption

Given

$$m \in \mathbb{Z}_N,$$

compute

$$c = m^e \bmod N.$$

using Bob's public key.

Decryption

Recover

$$m = c^d \bmod N.$$

using Bob's secret key.

The correctness follows from

$$(m^e)^d = m^{ed} \equiv m \pmod{N}.$$

Properties of RSA

Security Assumption

Recovering the secret key from the public key was believed to be as hard as factoring

$$N = pq.$$

Advantages

- Public-key encryption.
- Digital signatures.
- Simple mathematical construction.

Limitation

Large-scale quantum computers running Shor's algorithm can efficiently factor N . Consequently, RSA is not considered post-quantum secure and is being replaced by ML-KEM for key establishment and ML-DSA for digital signatures.

Key-Pair Generation ElGamal

ElGamal Public-Key Encryption: Key Generation

Public parameters

$$(G, q, g),$$

where G is a cyclic group of order q generated by g .

- 1 Choose a secret key

$$x \xleftarrow{\$} \mathbb{Z}_q.$$

- 2 Compute the public key

$$y = g^x.$$

$$pk = (G, q, g, y), \quad sk = x.$$

Security Assumption

Security relies on the hardness of the **Discrete Logarithm Problem (DLP)**.

ElGamal Encryption and Decryption

Encryption

Given message

$$m \in G,$$

choose random

$$r \xleftarrow{\$} \mathbb{Z}_q.$$

Compute

$$c_1 = g^r,$$

$$c_2 = my^r.$$

Ciphertext

$$c = (c_1, c_2).$$

Decryption

Compute

$$c_1^x = g^{rx} = y^r.$$

Recover

$$m = \frac{c_2}{c_1^x}.$$

Properties of ElGamal Encryption

Advantages

- Based on the discrete logarithm problem.
- Naturally randomized using the fresh exponent r .
- Foundation for many modern cryptographic constructions.

Limitations

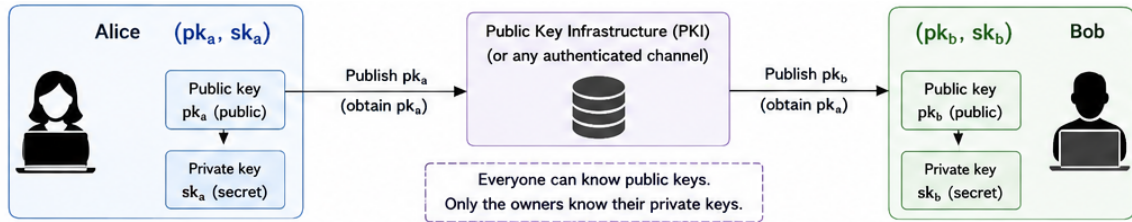
- Ciphertext consists of two group elements.
- Requires fresh randomness for every encryption.
- Broken by Shor's algorithm on a large-scale quantum computer.

Observation

RSA is based on integer factorization, whereas ElGamal is based on the discrete logarithm problem. Both are classical public-key encryption schemes and both are replaced by post-quantum KEMs in modern standards.

Publishing Public-key

Publishing Public-key



From Public Keys to Trusted Public Keys

Using ElGamal (or RSA), every entity generates

(pk, sk) .

The secret key sk is kept private, while the public key pk is published.

$pk \Rightarrow$ Public Directory

Problem

How does Alice know that the published public key really belongs to Bob?

An attacker may replace Bob's public key with its own, leading to a **Man-in-the-Middle (MITM)** attack.

Solution

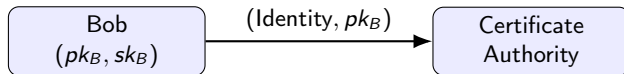
Instead of trusting the public key directly, trust a **Digital Certificate** issued by a trusted **Certificate Authority (CA)**.

Certificate Generation by a Certificate Authority

Suppose Bob has generated

$$(pk_B, sk_B).$$

Bob submits his identity and public key to a trusted Certificate Authority (CA).



The CA verifies Bob's identity and computes

$$\sigma = \text{Sign}_{sk_{CA}}(\text{Identity} \parallel pk_B).$$

The resulting certificate is

$$\text{Cert}_B = (\text{Identity}, pk_B, \sigma).$$

Certificate Verification

Alice receives

$$\text{Cert}_B = (\text{Identity}, pk_B, \sigma).$$

She already knows the CA's public key

$$pk_{CA}.$$

Alice verifies

$$\text{Verify}_{pk_{CA}}(\sigma) = 1.$$

If the verification succeeds,

pk_B is authentic.

Alice can now safely encrypt

$$c = \text{Enc}_{pk_B}(m),$$

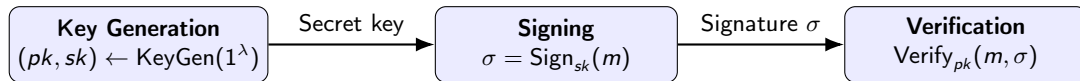
without fear of a Man-in-the-Middle attack.

Digital Signature

Digital Signature Scheme

A digital signature scheme consists of three polynomial-time algorithms

$(\text{KeyGen}, \text{Sign}, \text{Verify})$.



Security Goal

Only the owner of the secret key can generate a valid signature, while anyone possessing the public key can verify it.

ElGamal Digital Signature

Public parameters

$$(G, q, g).$$

Key generation

$$x \xleftarrow{\$} \mathbb{Z}_q, \quad y = g^x.$$

To sign a message m ,

1 Choose random

$$k \xleftarrow{\$} \mathbb{Z}_q, \quad \gcd(k, q) = 1.$$

2 Compute

$$r = g^k.$$

3 Compute

$$s = k^{-1} (H(m) - xr) \bmod q.$$

4 Output

$$\sigma = (r, s).$$

Verification of ElGamal Signature

Given

$$(m, r, s),$$

compute

$$v_1 = g^{H(m)},$$

and

$$v_2 = y^r r^s.$$

Accept iff

$$v_1 = v_2.$$

Security Assumption

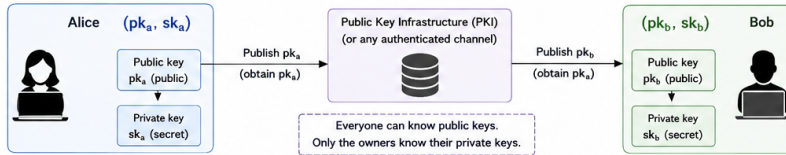
The security of ElGamal signatures relies on the hardness of the Discrete Logarithm Problem (DLP).

The random nonce k must never be reused.

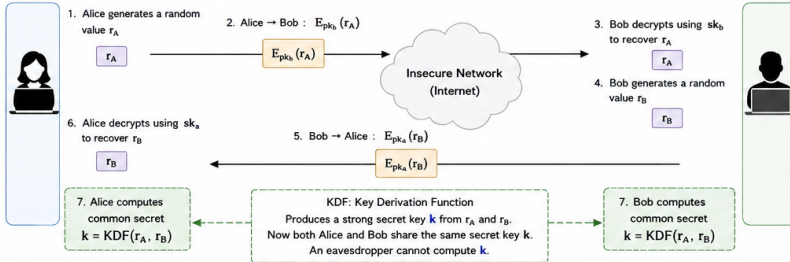
Revisiting the Full Picture

Secure Communication: a Full Picture

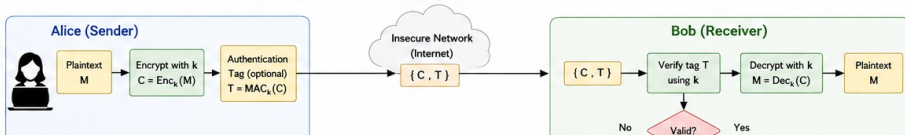
- 1) Each party has a public/private key pair



- 2) Establish a common secret key k



- 3) Use the common secret key k for secure communication



Fundamental Cryptographic Primitives

Primitive	Purpose	Examples
Block Cipher	Confidentiality for fixed-size blocks; used with modes of operation for long messages	AES, SM4
Stream Cipher	Confidentiality by generating a pseudorandom keystream	ChaCha20, Salsa20
Hash Function	Fixed-length digest for integrity and fingerprinting	SHA-256, SHA-3
Public-Key Encryption	Securely transmit secrets without a pre-shared key	RSA, ElGamal, ML-KEM
Digital Signature	Authentication, integrity, and non-repudiation	DSA, ECDSA, ML-DSA

Observation

Modern cryptographic protocols are built by combining these primitives. For example,

Public-Key Encryption → Key Exchange → Session Key → Block/Stream Cipher + AEAD.

Textbook:

- 1 Douglas R. Stinson and Maura Paterson. *Cryptography: Theory and Practice*. 4th Edition. Chapman & Hall/CRC Press, 2019. ISBN: 9780367148591.

Acknowledgment:

- The figures in this work were generated with the assistance of ChatGPT (OpenAI).



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.